WeaveNYC

Privacy-First Mutual Aid Platform

Connecting NYC's underserved communities with resources and neighbors they can trust

Kairan Shen · Software Engineering · Spring 2026

- **Local-First Storage**
  Data stays on device by default
- **P2P via GossipSub**
  Decentralized community sync
- **Progressive Auth**
  4-level identity trust model
- **Multi-Cloud (×3)**
  AWS + GCP + Azure PaaS

1.5M+ food insecure in NYC

2.5M limited English proficiency

300K+ undocumented at risk

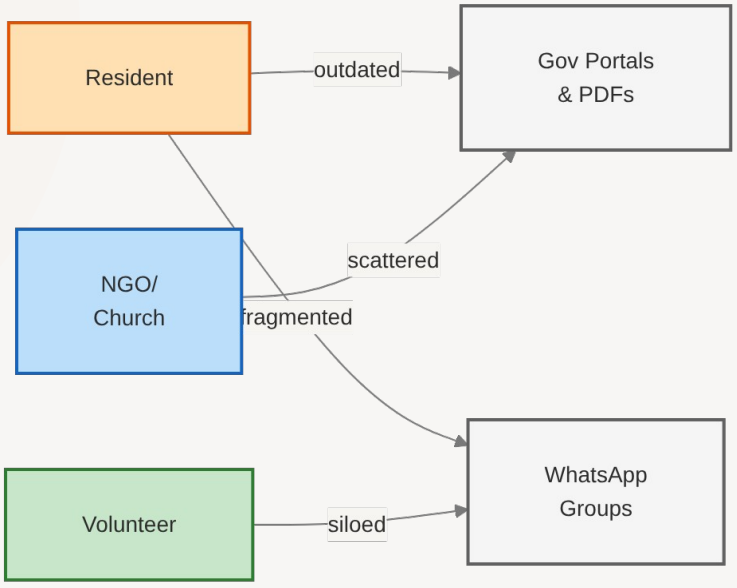
Problem: Fragmented, Opaque, Siloed

Fragmented Resources

Official info scattered across agencies, PDFs, and phone lines

Language & Digital Barriers

200+ languages in NYC, low digital literacy, poor connectivity



Current As-Is Flow: Manual, fragmented, no single system of record

Trust Deficit

Undocumented populations fear data exposure to authorities

Siloed Mutual Aid

WhatsApp/WeChat groups: limited reach, unsearchable, no governance

Solution: Five Value Pillars

01

Zero Cold Start

Official resource directory available Day 1 — no community bootstrapping needed

02

Privacy by Default

Data stays on device; user explicitly opts in before any sharing; ZIP-only precision

03

Smart Matching

Every request gets Top-3 action plans: neighbor matches + official resource fallback

04

Community Governance

Anchor nodes define rules, moderate content, broadcast decisions via P2P network

05

Trilingual + Voice

Full UI in EN/ES/ZH; 3-tier voice degradation; privacy-first audio processing

Target Users

Three personas across the WeaveNYC platform

Need Seeker

WHO

Low-income, immigrant, elderly, single parent, temporarily unemployed

GOAL

Find food, childcare, translation, transportation help ASAP

OUR VALUE

Actionable plan in <1 min; anonymous by default; minimal data exposure

Helper

WHO

Volunteer, neighbor, student, church / community group member

GOAL

Post offers, screen suitable matches, complete delivery safely

OUR VALUE

Smart matching; risk alerts; witness mechanism builds trust

Anchor Admin

WHO

NGO staff, church coordinator, pantry operator, mutual aid organizer

GOAL

Configure rules, moderate content, post announcements, handle reports

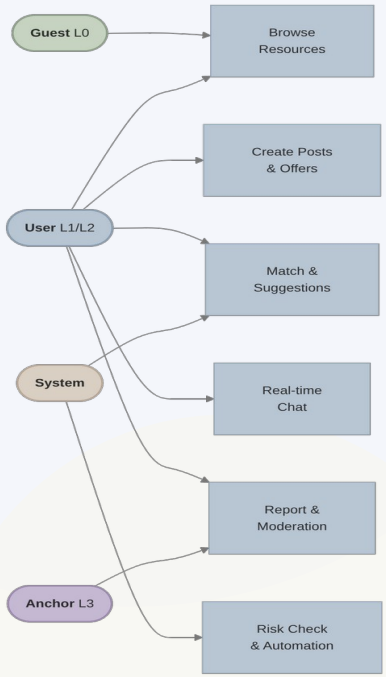
OUR VALUE

Visual review queue + audit log; community governance toolkit

Seeker requests help → Helper fulfills → Anchor Admin governs

MVP Use Cases

Five core workflows from browse to governance



01 **Browse Resources**

ZIP search, 'open now' filter, one-tap call/navigate

02 **Post Request / Offer**

Voice or text, category, privacy level, auto-expire

03 **Smart Matching**

Top-3 action plans: distance, language, trust signals

04 **Safe Communication**

Double opt-in chat, PII detection, block/report

05 **Anchor Governance**

Review queue, SLA priority, audit trail, P2P broadcast

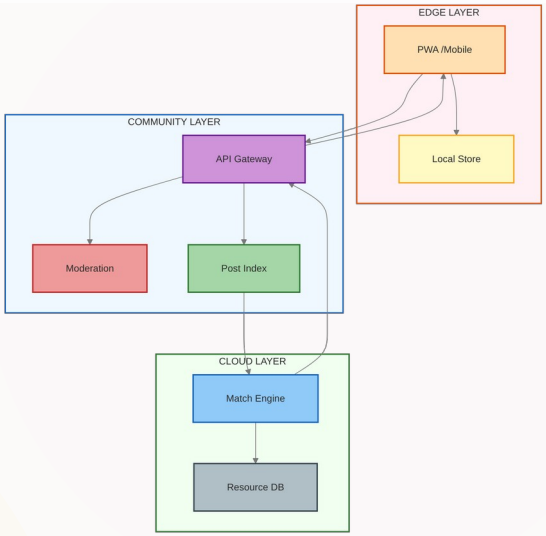
System Architecture

Four-layer separation from client to storage

Edge Layer

PWA / Client

Next.js + IndexedDB + i18n
Map + Voice; offline-first



Community Layer

P2P / Anchor

libp2p + Anchor Node
GossipSub + IPFS pin + mod

Cloud Layer

6 Microservices

API GW, Resource Dir, Posts
Match Engine, Chat, Notify

Data Layer

Storage

PostgreSQL+PostGIS | Redis
IPFS/Helia | S3/GCS/Blob

P2P Architecture: Node Complementarity

How a post travels through the governance-layered P2P network

STAGE 1: CREATE

Edge Node

PWA / Browser (libp2p-js)

- Local-first storage (IndexedDB)
- libp2p in WebWorker (non-blocking)
- WebRTC direct + WebSocket relay
- Max 10 concurrent connections
- Ed25519 keypair + event signing

Users create posts offline, opt-in to publish PostSummary to GossipSub

publish event

STAGE 2: RELAY & GOVERN

Anchor Node

Community Org (Node.js, libp2p v1.1)

- GossipSub relay + broadcaster
- IPFS Helia pin (30-day TTL)
- Bootstrap: last-100 post snapshots
- Content moderation + rule engine
- Signature verification + audit log

Validates, relays events; Anchor-wins for moderation decisions

relay / compute

STAGE 3: COMPUTE

Cloud Intelligence

AWS / GCP / Azure (Serverless)

- Optional relay (Anchor offline fallback)
- Match engine: Top-3 + official fallback
- Risk detection + speech recognition
- Geo-indexed search (PostGIS)
- No persistent personal data stored

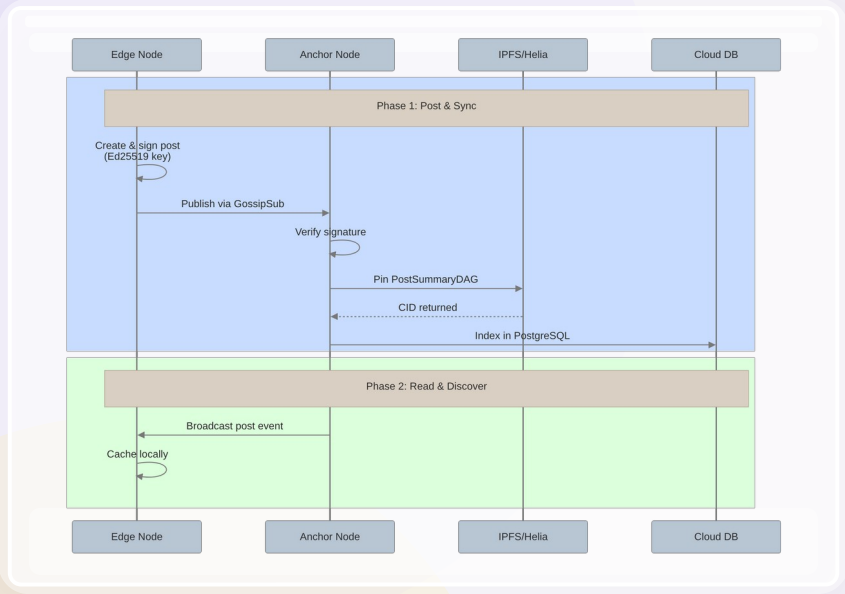
Heavy compute only; indexes metadata, never raw chat or PII

Not fully decentralized — a governance-layered hybrid: Edge provides privacy, Anchor provides trust, Cloud provides intelligence. DHT disabled; Anchor indexes instead.

Privacy First | **Community Self-Governance** | **P2P ≠ Kafka** | **Resilient Mesh**

P2P Sync Protocol & Consistency

GossipSub event-driven communication with eventually-consistent model



P2P Sync Sequence Diagram

GossipSub Event Protocol

/posts create / update / delete

/reports flags (min evidence)

announcement Anchor notices

Event Envelope: event_id (UUIDv7) + type + community_id + actor (Ed25519) + ts + payload + signature

Bootstrap: peer → Anchor → last-100 snapshot (Ed25519 verified) → subscribe 3 topics → incremental sync (~500ms)

Consistency & Conflict Resolution

Event Dedup
UUIDv7 unique ID
drop duplicates

Version Chain
CID → prev_cid
immutable audit

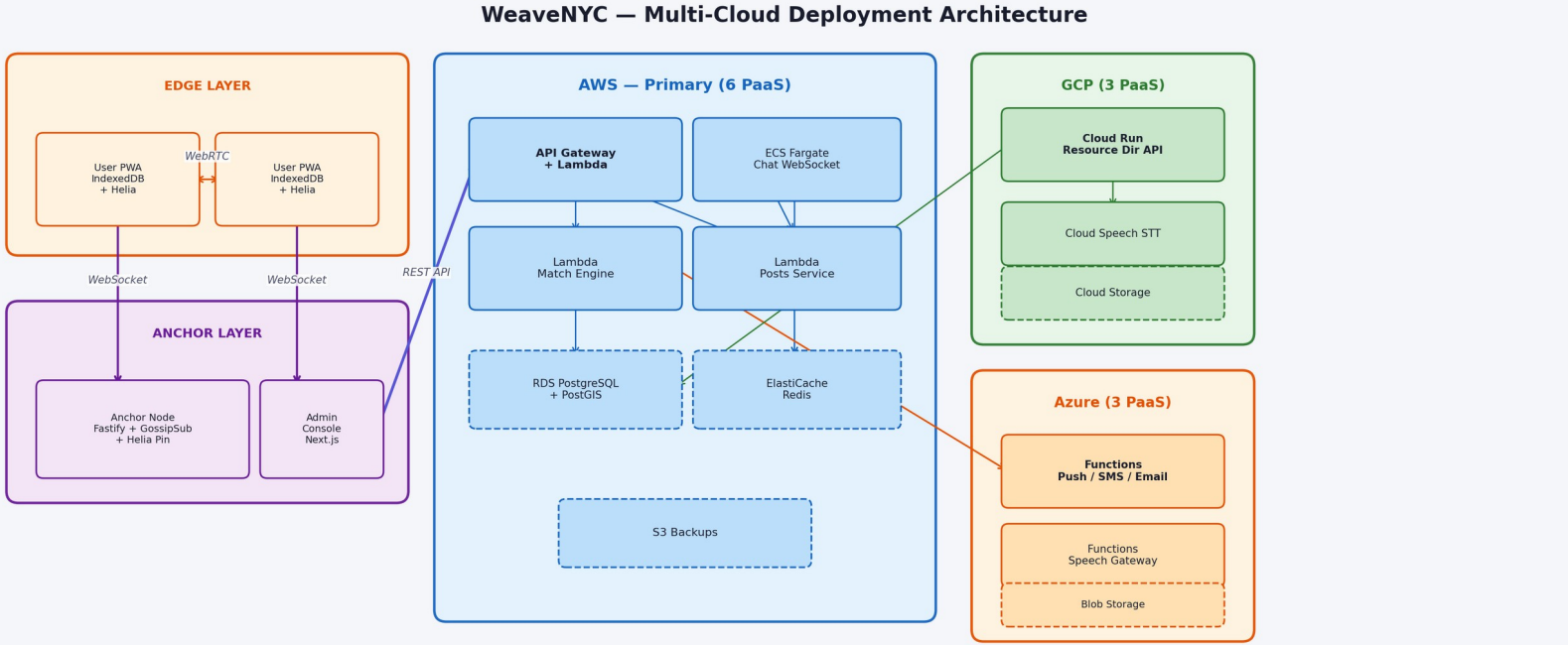
Conflict Rule
Posts: last-write-wins
Mod: Anchor-wins

Tombstone
Irreversible delete
broadcast to all

libp2p + Helia | GossipSub v1.1 | Ed25519 | WebRTC + WS | IndexedDB

Multi-Cloud Deployment

12 PaaS services across AWS, GCP & Azure — 3-tier architecture with strategic workload distribution



Why Multi-Cloud? Each provider chosen for a specific technical advantage

AWS

Mature serverless stack (Lambda+API GW) for sub-100ms API response. PostGIS for geo-fenced search. Fargate for persistent WS connections.

GCP

Cloud Run auto-scales to zero — isolates Resource Directory from main API blast radius. Best price/perf for bursty traffic.

Azure

SignalR-grade push notification hub. Azure Speech = lowest-latency STT for Tier-3 voice features. Blob = cross-cloud DR backup.

Data Model & Privacy Tiers

Local-Only

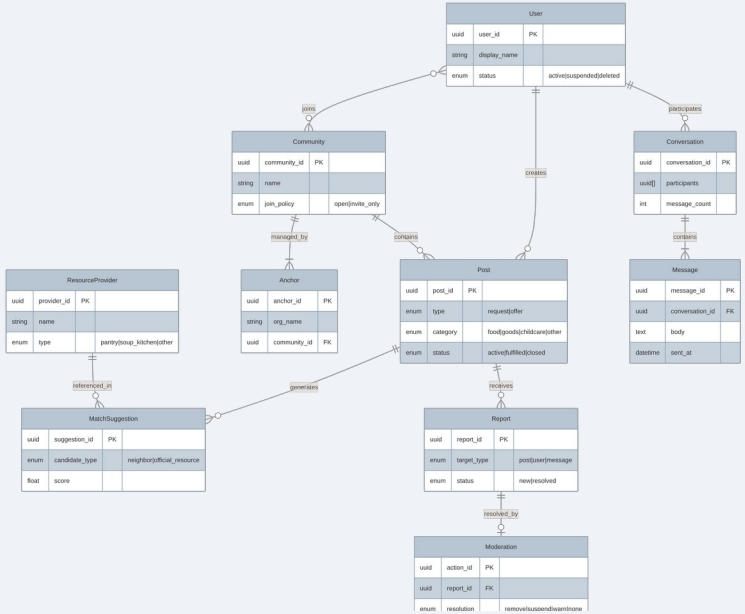
IndexedDB (Dexie.js): settings, drafts, favorites, keys. Data never leaves device.

Community-Shared

IPFS (Helia) + GossipSub: post summaries, version chains, reputation signals.

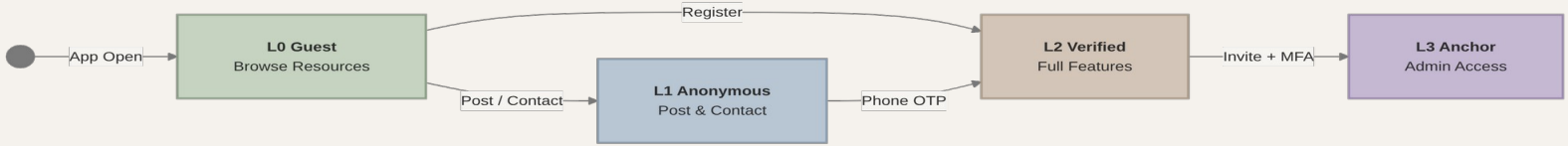
Cloud Index

PostgreSQL+PostGIS + Redis: resources, post indexes, match results, chat, audit logs.



ER diagram with three-tier storage architecture

Security & Trust



Threat Model (STRIDE)

Spoofing

Anchor MFA; signed resource directory

Tampering

Import version hash; post version chain

Repudiation

Minimal audit log (type + time + actor)

Info Disclosure

ZIP-only; no GPS; anonymous nicknames

DoS

Rate limit; new-account restrictions; topic quota

Elevation

RBAC + invite code + MFA for Anchor

Community Governance: Reports → Moderation → Anchor Decision → P2P Broadcast

Key Engineering Decisions (ADRs)

ADR	Decision	Why
ADR-001	**Default Local Storage**	Three-layer: IndexedDB → IPFS → Postgres
ADR-002	**Anchor Final Authority**	P2P needs trusted governance
ADR-003	**Always-Fallback Matching**	Emergency never sees empty results
ADR-004	**Full-stack TypeScript**	Monorepo + shared Zod schemas
ADR-005	**No Token / No Payment**	Avoids legal complexity
ADR-006	**ZIP-Only Precision**	Privacy for undocumented users
ADR-007	**Eventual Consistency**	Posts: last-write-wins; Moderation: Anchor-wins
ADR-008	**Voice 3-Tier Degradation**	Device → Web Speech → Cloud Gateway

Roadmap & Course Alignment

M0 *Day 1-3*

Project Skeleton

Monorepo + CI/CD + DB + JWT + PWA

M1 *Wk 1-2*

Resource Directory

CFC import + ZIP search + PostGIS

M2 *Wk 2-3*

Mutual Aid Posting

Request/Offer CRUD + voice input

M3 *Wk 3-4*

Matching Engine

Top-3 action plans + official fallback

M4 *Wk 4-5*

Chat & Safety

WebSocket RT + PII detect + mod queue

M5 *Wk 5-6*

Anchor Admin

Review queue + announcements + rules

M6 *Wk 6+*

P2P Sync (Bonus)

Edge IPFS + Anchor pin + conflict resolution

- ✓ Multi-Cloud (≥2): AWS + GCP + Azure
- ✓ PaaS (≥3): Lambda, Cloud Run, Azure Funcs, ECS...
- ✓ P2P/Decentralized: libp2p + Helia + GossipSub
- ✓ Security: 4-level auth, STRIDE, Anchor moderation
- ✓ Demo + Report: 7 milestones → live demo

Ready to Build

7 milestones across 6+ weeks

20 docs | 14 diagrams | Full API + DB spec

MVP-first, iterate on feedback